DOCUMENTACIÓN DE ARQUITECTURA

SISTEMA DE RESERVA DE PISTAS DEPORTIVAS

Proyecto de portfolio

José Ignacio Segovia Ramírez

Versión 1.0

Septiembre de 2026

Índice

1. Introducción y objetivos....5
1.1. Requisitos....5
1.1.1. Propósito y alcance....5
1.1.2. Actores y tipos de usuario....5
1.1.3. Requisitos funcionales....5
1.1.4. Documento de requisitos....6
1.2. Objetivos de calidad....7
1.3. Stakeholders....7
2. Restricciones arquitectónicas....8
2.1. Tecnologías y frameworks....8
2.2. Infraestructura y despliegue....9
2.3. Persistencia de datos....9
2.4. Almacenamiento de archivos....9
2.5. Autenticación y sesiones....10
2.6. Caché....10
2.7. Procesamiento de tareas en segundo plano....10
2.8. Gestión de pagos....11
2.9. Envío de correo electrónico....11
2.10. Seguridad y configuración....11
2.11. Control de versiones y despliegue continuo....11
2.12. Dependencias externas....12
3. Contexto y ámbito de aplicación....12
3.1. Contexto de negocio....12
3.2. Contexto técnico....13
3.3. Alcance del sistema....14
3.4. Entorno de despliegue....15
4. Estrategia de solución....16
4.1. Arquitectura MVC....16
4.2. Separación de responsabilidades....16
4.3. Presentación mediante Blade y Livewire....17
4.4. Persistencia mediante PostgreSQL y Eloquent....17
4.5. Autenticación y autorización....18
4.6. Gestión de la lógica de negocio....18
4.7. Integración con Stripe....19
4.8. Gestión de tareas mediante colas....19
4.9. Gestión de sesiones y almacenamiento....19
4.10. Despliegue mediante Docker....20
4.11. Envío de correo electrónico....20
4.12. Resumen de la estrategia....20
5. Vista de bloques de construcción....21

5.1. Nivel 1: Sistema....21
5.2. Nivel 2: Bloques principales....22
5.3. Presentación y entrada al sistema....23
5.3.1. Controllers....23
5.3.2. Livewire....24
5.3.3. Blade y componentes de vista....24
5.4. Lógica de aplicación y negocio....24
5.5. Persistencia....26
5.5.1. Concerns de los modelos....26
5.6. Validación y autorización....26
5.6.1. Form Requests....26
5.6.2. Rules....27
5.6.3. Policies....27
5.6.4. Middleware....27
5.7. Filtrado y consulta de datos....27
5.8. Integración con Stripe....28
5.9. Comandos....28
5.10. Enums....28
5.11. Excepciones....29
5.12. Helpers....29
5.13. Providers....29
5.14. Correo electrónico....30
6. Vista de ejecución....30
6.1. Realización de una reserva....31
6.2. Recepción de un webhook de Stripe....31
6.3. Cancelación de una reserva....32
6.4. Cancelación automática de reservas expiradas....32
6.5. Resumen de los flujos de ejecución....33
7. Vista de despliegue....34
7.1. Infraestructura general....34
7.2. Código fuente y proceso de despliegue....34
7.3. Contenedor de aplicación....34
7.4. Servidor web....35
7.5. Base de datos....35
7.6. Persistencia del almacenamiento local....35
7.7. Sesiones, caché y colas....36
7.8. Servicio de correo....36
7.9. Integración con Stripe....36
7.10. Entorno de despliegue....37
7.11. Resumen....37
8. Conceptos transversales....37
8.1. Autenticación y autorización....38

8.2. Validación de datos....38
8.3. Gestión de errores y excepciones....39
8.4. Persistencia de datos....39
8.5. Gestión de reservas....40
8.6. Integración con Stripe....41
8.7. Envío de correos....41
8.8. Sesiones, caché y colas....42
8.9. Interfaz de usuario....42
8.10. Organización de responsabilidades....43
8.11. Presentación e interacción con el usuario....43
9. Decisiones arquitectónicas....44
9.1. Utilización de Laravel....44
9.2. Utilización de una arquitectura MVC....44
9.3. Utilización de Services....45
9.4. Utilización de PostgreSQL....45
9.5. Externalización de la base de datos en Neon....46
9.6. Utilización de Docker para el despliegue....46
9.7. Utilización de Render como plataforma de despliegue....46
9.8. Utilización de Blade y Livewire....46
9.9. Utilización de Stripe para los pagos....47
9.10. Separación de validación y autorización....47
9.11. Uso de mecanismos de Laravel para tareas transversales....47
10. Requisitos de calidad....48
10.1. Mantenibilidad....48
10.2. Seguridad....49
10.3. Fiabilidad....50
10.4. Rendimiento....50
10.5. Usabilidad....51
10.6. Desplegabilidad....51
11. Riesgos y deuda técnica....52
11.1. Configuración del correo electrónico....52
11.2. Almacenamiento local del contenedor....52
11.3. Almacenamiento de sesiones....53
11.4. Caché y colas mediante PostgreSQL....53
11.5. Dependencia de servicios externos....54
11.6. Ausencia de objetivos de disponibilidad....54
11.7. Ausencia de pruebas de carga....55
11.8. Escalabilidad....55
11.9. Monitorización....56
11.10. Deuda técnica y mejoras futuras....56
11.11. Resumen....57
12. Glosario....58

12.1. Convenciones utilizadas en el código.....59

1. Introducción y objetivos

1.1. Requisitos

1.1.1. Propósito y alcance

El proyecto consiste en una aplicación web que gestiona las reservas online de las pistas del polideportivo y de la ciudad deportiva en Moral de Calatrava.

Actualmente, la única forma de reservar las pistas del polideportivo o de la ciudad deportiva en Moral de Calatrava es ir en persona. Con esta aplicación, la gente tendrá la opción de reservar a distancia en cualquier momento.

1.1.2. Actores y tipos de usuario

Hay tres perfiles de usuario identificados en el sistema:

1. Cliente. Es el usuario que utiliza el sistema. Puede realizar reservas, cancelarlas, consultar el historial de reservas realizadas, modificar sus datos personales, enviar comentarios y consultar el historial de comentarios.
2. Gestor. Gestiona la aplicación. Puede crear, modificar y borrar pistas; consultar, modificar y cancelar las reservas de los clientes; crear, modificar y borrar horarios no disponibles para los clientes (por ejemplo, para realizar labores de mantenimiento); y consultar los comentarios enviados por los clientes.
3. Administrador. Administra los datos de los gestores. Es también un gestor (así que puede hacer todo lo que hace un gestor). Puede crear, modificar y eliminar gestores (excepto a sí mismo).

1.1.3. Requisitos funcionales

En el documento de requisitos se encuentra una descripción más detallada de los requisitos, disponible en docs/requirements/requirements-sports-court-booking.pdf.

Usuarios no autenticados:

1. Consultar las pistas y sus horarios disponibles.
2. Registrarse como cliente.

Clientes:

1. Consultar y/o modificar sus datos personales.
2. Realizar reservas.
3. Consultar el historial de reservas.
4. Anular reservas.
5. Enviar comentarios.
6. Consultar el historial de comentarios.

Gestores:

1. Consultar los datos de las reservas realizadas.
2. Modificar y cancelar las reservas realizadas por los clientes.
3. Consultar la lista de reservas realizadas en una pista.
4. Consultar el calendario con las reservas realizadas en una pista.
5. Crear, modificar y eliminar pistas.
6. Elegir las fechas que estarán disponibles para reservar las pistas.
7. Consultar los comentarios enviados por los clientes.

Administradores:

1. Gestionar los gestores del sistema (crear, consultar, modificar y eliminar).

1.1.4. Documento de requisitos

Los requisitos funcionales y no funcionales se encuentran detallados en el documento de requisitos, donde se especifican los requisitos del sistema y los casos de uso asociados, disponible en docs/requirements/requirements-sports-court-booking.pdf.

1.2. Objetivos de calidad

Prioridad	Objetivo de calidad	Motivación / descripción
1	Integridad de reservas	Nunca deben existir dos reservas confirmadas para la misma pista y franja horaria, incluso con peticiones concurrentes
2	Mantenimiento y tests	Las reglas de negocio deben estar aisladas y cubiertas por tests automatizados, de modo que añadir una regla nueva (p. ej. política de cancelación) no requiera tocar más de un módulo
3	Usabilidad	Un usuario sin formación previa debe poder completar una reserva desde el móvil en pocos pasos
4	Seguridad	Cada usuario solo accede a sus propios datos y reservas; los flujos de pago simulados no almacenan datos reales de tarjeta
5	Facilidad de despliegue	El proyecto debe poder ejecutarse localmente con datos de ejemplo en menos de 10 minutos siguiendo el README

1.3. Stakeholders

Stakeholder	Interés / responsabilidad
Autor/desarrollador	Diseñar, implementar, probar y mantener la aplicación
Evaluador técnico / reclutador	Evaluar las capacidades técnicas demostradas mediante el proyecto.
Usuario potencial	Representar los perfiles que utilizarían la aplicación en un escenario real y servir como referencia para el diseño funcional.

La aplicación se desarrolla como proyecto de portfolio con el objetivo de demostrar las capacidades técnicas y de diseño del autor. No está destinada a prestar actualmente un servicio real ni cuenta con usuarios finales reales.

El autor del proyecto actúa simultáneamente como desarrollador y responsable de las decisiones funcionales y técnicas. Los perfiles de cliente, gestor y administrador representan usuarios potenciales del sistema en un escenario de uso real.

La documentación y la implementación están orientadas a facilitar la evaluación del proyecto por parte de terceros, especialmente profesionales técnicos que puedan valorar las decisiones de diseño, arquitectura, implementación y calidad del software.

2. Restricciones arquitectónicas

Las restricciones arquitectónicas son aquellas condiciones tecnológicas, de infraestructura y de servicios externos que condicionan las decisiones de diseño y la implementación de la aplicación.

2.1. Tecnologías y frameworks

La aplicación está desarrollada utilizando las siguientes tecnologías:

1. Backend: PHP 8.4.
2. Framework: Laravel Framework 12.62.0.
3. Frontend: Blade y Livewire.
4. Gestión y compilación de recursos frontend: Vite.
5. Servidor web: Nginx.
6. Base de datos: PostgreSQL.
7. Contenerización: Docker.

El desarrollo sobre Laravel condiciona la arquitectura de la aplicación al modelo y mecanismos proporcionados por el framework para funcionalidades como routing, autenticación, gestión de sesiones, acceso a datos, validación, gestión de colas, caché, envío de correo y gestión de archivos.

La interfaz de usuario utiliza Blade y Livewire, por lo que la capa de presentación se basa en los mecanismos de renderizado y comunicación proporcionados por estas tecnologías.

Los recursos frontend son gestionados y compilados mediante Vite.

2.2. Infraestructura y despliegue

La aplicación está desplegada como un contenedor Docker en la plataforma Render.

El código fuente se encuentra alojado en GitHub y el repositorio está conectado con Render. La rama master está configurada para realizar despliegues automáticos cuando se producen cambios en el repositorio.

La construcción de la aplicación en Render se realiza utilizando el Dockerfile definido específicamente para el despliegue:

docker/php/Dockerfile.render

Por tanto, la aplicación debe poder ser construida y ejecutada correctamente dentro de este entorno Docker y ser compatible con la infraestructura proporcionada por Render.

La aplicación se encuentra desplegada en la región de Frankfurt (EU Central).

2.3. Persistencia de datos

La aplicación utiliza PostgreSQL como sistema gestor de base de datos.

La base de datos no se encuentra alojada en Render, sino en Neon, por lo que la aplicación debe establecer una conexión de red con un servicio PostgreSQL externo a la infraestructura de ejecución de la aplicación.

Los parámetros de conexión y las credenciales de la base de datos se proporcionan mediante variables de entorno y no forman parte del código fuente.

La base de datos no solo almacena los datos propios de la aplicación, sino que también se utiliza para determinados mecanismos internos de Laravel, como la caché y las colas de trabajos.

2.4. Almacenamiento de archivos

La aplicación utiliza el sistema de archivos local proporcionado por Laravel, mediante:

FILESYSTEM_DISK=local

El servicio de Render no dispone de un disco persistente asociado al Web Service. Por este motivo, el almacenamiento local del contenedor no debe considerarse un mecanismo de persistencia permanente.

Los archivos que deban conservarse de forma permanente no deben depender exclusivamente del almacenamiento local del contenedor, ya que pueden perderse cuando la instancia sea reemplazada o se realice un nuevo despliegue.

2.5. Autenticación y sesiones

La autenticación de usuarios se implementa mediante los mecanismos proporcionados por Laravel.

Las sesiones de usuario utilizan el controlador de archivos, configurado mediante:

SESSION_DRIVER=file

Por tanto, los datos de sesión se almacenan en el filesystem local del contenedor.

Dado que el servicio de Render no dispone actualmente de almacenamiento persistente asociado, las sesiones almacenadas localmente no deben considerarse persistentes ante el reemplazo de la instancia.

Además, esta configuración condiciona una posible ampliación futura a múltiples instancias, ya que las sesiones almacenadas localmente no estarían compartidas entre diferentes instancias de la aplicación.

2.6. Caché

La aplicación utiliza el sistema de caché de Laravel con almacenamiento en base de datos:

CACHE_STORE=database

Por tanto, los datos de caché se almacenan en PostgreSQL, en lugar de depender del filesystem local o de un servicio externo de caché como Redis.

Esta decisión hace que la aplicación dependa de la disponibilidad de la base de datos PostgreSQL para determinadas operaciones de caché.

2.7. Procesamiento de tareas en segundo plano

La aplicación utiliza el sistema de colas de Laravel con almacenamiento en base de datos:

QUEUE_CONNECTION=database

Los trabajos pendientes se almacenan en PostgreSQL y son procesados por un worker de Laravel.

La arquitectura de despliegue debe garantizar la ejecución del proceso encargado de consumir y procesar estos trabajos cuando la aplicación utilice tareas asíncronas.

2.8. Gestión de pagos

La aplicación utiliza Stripe como pasarela de pago.

La integración con Stripe constituye una dependencia externa de la aplicación y condiciona la gestión del proceso de pago a las interfaces y mecanismos proporcionados por este servicio.

Las credenciales y claves necesarias para la comunicación con Stripe se gestionan mediante variables de entorno y no se almacenan directamente en el código fuente.

2.9. Envío de correo electrónico

La aplicación utiliza el sistema de correo proporcionado por Laravel y define los mensajes mediante clases Mailable.

Actualmente, el transporte de correo está configurado mediante:

MAIL_MAILER=log

Por tanto, los correos generados por la aplicación no se envían actualmente a destinatarios externos, sino que se registran en los logs de la aplicación.

La incorporación futura de un servicio de correo externo requerirá modificar la configuración del transporte y constituirá una nueva dependencia externa de la aplicación.

2.10. Seguridad y configuración

Las credenciales, claves de acceso y demás información sensible necesaria para la ejecución de la aplicación se gestionan mediante variables de entorno.

La configuración dependiente del entorno no debe estar incluida directamente en el código fuente ni en la imagen Docker.

La aplicación se encuentra publicada mediante HTTPS a través de la infraestructura de Render.

2.11. Control de versiones y despliegue continuo

El código fuente se gestiona mediante Git y se encuentra alojado en GitHub.

GitHub está integrado con Render para automatizar el proceso de despliegue. Los cambios realizados en la rama configurada para producción desencadenan la construcción y despliegue de una nueva versión de la aplicación.

El flujo de despliegue es:

GitHub → Render → construcción de imagen Docker → despliegue del contenedor

2.12. Dependencias externas

La arquitectura de la aplicación depende actualmente de los siguientes servicios externos:

1. Neon: proporciona la base de datos PostgreSQL utilizada por la aplicación.
2. Stripe: proporciona los servicios relacionados con el procesamiento de pagos.
3. Render: proporciona la infraestructura de ejecución, despliegue y publicación de la aplicación.
4. GitHub: proporciona el alojamiento del código fuente y su integración con el proceso de despliegue.

Actualmente, el envío de correo no constituye una dependencia de un servicio externo, ya que el transporte está configurado mediante MAIL_MAILER=log.

La utilización de PostgreSQL como almacenamiento de datos, caché y colas implica una dependencia transversal de la aplicación respecto a la disponibilidad de la base de datos proporcionada por Neon.

3. Contexto y ámbito de aplicación

La descripción de los perfiles de usuario y sus funcionalidades se encuentra en los apartados 1.1.2. Actores y tipos de usuario y 1.1.3. Requisitos funcionales, por lo que no se vuelve a detallar dicha información en este apartado.

3.1. Contexto de negocio

La aplicación tiene como finalidad gestionar la reserva de pistas deportivas y las operaciones relacionadas con dichas reservas.

El sistema permite a los usuarios consultar la disponibilidad de las pistas y realizar las operaciones correspondientes a su perfil. Los usuarios que interactúan directamente con la aplicación son los definidos en el apartado 1.1.2. Actores y tipos de usuario.

Además de estos actores, la aplicación se relaciona con sistemas externos necesarios para proporcionar determinadas funcionalidades.

Los principales sistemas externos son:

1. Stripe: servicio externo utilizado como pasarela de pago para gestionar los pagos asociados a las reservas.
2. Neon: servicio que proporciona la base de datos PostgreSQL utilizada por la aplicación para la persistencia de los datos.

3.2. Contexto técnico

La aplicación está implementada utilizando Laravel y se ejecuta sobre PHP 8.4. La interfaz web utiliza Blade y Livewire, y los recursos frontend son gestionados mediante Vite.

La aplicación se ejecuta dentro de un contenedor Docker desplegado en Render y utiliza Nginx como servidor web.

La comunicación con la base de datos se realiza mediante una conexión desde la aplicación desplegada en Render hacia la instancia PostgreSQL proporcionada por Neon.

Para las operaciones de pago, la aplicación se comunica con Stripe mediante los mecanismos de integración proporcionados por dicho servicio.

3.3. Alcance del sistema

El sistema comprende la aplicación web desarrollada con Laravel y las funcionalidades necesarias para la gestión de usuarios, pistas deportivas, disponibilidad, reservas, pagos y comentarios.

Dentro de los límites del sistema se encuentran:

1. La lógica de negocio de la aplicación.
2. La gestión de usuarios y sus perfiles.
3. La autenticación de usuarios.
4. La gestión de pistas deportivas.
5. La gestión de horarios y disponibilidad.
6. La gestión de reservas.
7. La integración con Stripe para el proceso de pago.

8. La gestión de comentarios.
9. La gestión de los datos utilizados por la aplicación.
10. Los mecanismos de caché y colas proporcionados por Laravel.

Quedan fuera del límite del sistema las responsabilidades correspondientes a servicios externos, entre ellas:

1. La infraestructura de base de datos proporcionada por Neon.
2. El procesamiento de pagos proporcionado por Stripe.
3. La infraestructura de ejecución y despliegue proporcionada por Render.
4. El alojamiento del código fuente proporcionado por GitHub.
5. El navegador utilizado por los usuarios.
6. La infraestructura de red e Internet.
7. La aplicación se integra con estos elementos, pero no controla su funcionamiento interno.

3.4. Entorno de despliegue

El código fuente de la aplicación se encuentra alojado en GitHub y está conectado con Render para realizar el despliegue de la aplicación.

El proceso de despliegue se basa en la construcción de una imagen Docker a partir del código fuente y su posterior ejecución en Render.

GitHub y Render forman parte del entorno de desarrollo y despliegue de la aplicación, mientras que Neon y Stripe forman parte del contexto externo con el que la aplicación interactúa durante su funcionamiento.

4. Estrategia de solución

La estrategia de solución describe las principales decisiones arquitectónicas adoptadas para satisfacer los requisitos del sistema y las restricciones identificadas.

4.1. Arquitectura MVC

La aplicación utiliza una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC), siguiendo las convenciones proporcionadas por Laravel.

Las responsabilidades principales se distribuyen entre:

1. Modelos: representan los datos y las entidades del sistema y gestionan su persistencia mediante Eloquent ORM.
2. Vistas: proporcionan la interfaz de usuario mediante Blade y componentes Livewire.
3. Controladores: reciben y procesan las peticiones HTTP y coordinan la ejecución de las operaciones correspondientes.

El patrón MVC permite separar la presentación, la gestión de las peticiones y el acceso a los datos, facilitando el mantenimiento y evolución de la aplicación.

La arquitectura MVC se complementa con otros mecanismos proporcionados por Laravel y con componentes propios de la aplicación para evitar concentrar toda la lógica en los controladores.

4.2. Separación de responsabilidades

La aplicación utiliza diferentes mecanismos de Laravel para distribuir las responsabilidades y evitar la concentración de lógica en los controladores.

Entre los principales componentes utilizados se encuentran:

1. Controllers: gestionan las peticiones HTTP y coordinan las operaciones.
2. Models: representan las entidades y gestionan su persistencia mediante Eloquent ORM.
3. Services: encapsulan operaciones y lógica de negocio que requieren coordinación entre diferentes componentes.
4. Commands: encapsulan operaciones que pueden ejecutarse mediante comandos de Laravel.
5. Requests: gestionan la validación y autorización asociada a las peticiones HTTP.
6. Rules: encapsulan reglas de validación específicas.
7. Policies: implementan las reglas de autorización sobre los recursos.
8. Middlewares: implementan lógica transversal durante el procesamiento de las peticiones.
9. Enums: representan conjuntos definidos de valores utilizados por la aplicación.

10. Exceptions: representan situaciones excepcionales específicas de la aplicación.
11. Helpers: proporcionan funcionalidades auxiliares reutilizables.
12. Providers: permiten registrar y configurar servicios y componentes de la aplicación.
13. Query Filters: encapsulan criterios de filtrado aplicados a las consultas.
14. Livewire: permite implementar componentes interactivos de la interfaz de usuario.

Estos mecanismos complementan la arquitectura MVC, pero no sustituyen el patrón arquitectónico principal.

4.3. Presentación mediante Blade y Livewire

La capa de presentación utiliza Blade como sistema de plantillas y Livewire para implementar componentes interactivos.

Esta estrategia permite desarrollar una interfaz dinámica manteniendo el frontend integrado en la aplicación Laravel, sin necesidad de desarrollar una aplicación frontend independiente.

Los recursos frontend son gestionados y compilados mediante Vite.

4.4. Persistencia mediante PostgreSQL y Eloquent

La persistencia de los datos se realiza mediante PostgreSQL, utilizando Eloquent ORM como mecanismo de acceso a datos proporcionado por Laravel.

La base de datos está externalizada en Neon.

Además de almacenar los datos propios del dominio, PostgreSQL se utiliza para determinados mecanismos internos de Laravel:

1. Caché mediante CACHE_STORE=database.
2. Almacenamiento de trabajos en cola mediante QUEUE_CONNECTION=database.

De esta forma, no es necesario incorporar servicios adicionales como Redis para estas funcionalidades.

4.5. Autenticación y autorización

La autenticación se implementa utilizando los mecanismos proporcionados por Laravel.

La autorización se gestiona mediante Policies, que permiten definir las operaciones que cada usuario puede realizar sobre los recursos del sistema.

La aplicación contempla tres perfiles de usuario:

1. Cliente.
2. Gestor.
3. Administrador.

Las responsabilidades y funcionalidades asociadas a cada perfil se encuentran descritas en los apartados 1.1.2. Actores y tipos de usuario y 1.1.3. Requisitos funcionales.

4.6. Gestión de la lógica de negocio

La lógica de negocio se distribuye entre diferentes componentes de la aplicación.

Los Services se utilizan para encapsular operaciones de negocio que requieren coordinación entre diferentes componentes o entidades. Los controladores delegan en estos servicios las operaciones que requieren una lógica de negocio más compleja, evitando concentrar dicha lógica en los controladores.

Las validaciones se gestionan mediante Form Requests y Rules, mientras que las restricciones de acceso a los recursos se implementan mediante Policies.

Los Enums permiten representar de forma explícita valores pertenecientes a conjuntos definidos por el dominio.

Las situaciones excepcionales específicas de la aplicación se representan mediante clases Exception.

Esta distribución busca mantener una separación clara de responsabilidades dentro de la arquitectura MVC.

4.7. Integración con Stripe

La aplicación utiliza Stripe como pasarela de pago.

La integración con Stripe se realiza mediante un controlador específico para recibir las notificaciones enviadas por Stripe mediante webhooks.

El controlador StripeWebhookController se encarga de recibir la petición, obtener el contenido y la firma enviada por Stripe y verificar la autenticidad del webhook.

Una vez validado el evento, el procesamiento se delega en StripeWebhookService, evitando incluir la lógica de negocio directamente en el controlador.

La comunicación con Stripe constituye una dependencia externa de la aplicación.

4.8. Gestión de tareas mediante colas

La aplicación utiliza el sistema de colas proporcionado por Laravel mediante:

QUEUE_CONNECTION=database

Los trabajos pendientes se almacenan en PostgreSQL.

Esta estrategia permite desacoplar determinadas operaciones del ciclo de vida de una petición HTTP y disponer de un mecanismo para ejecutar tareas de forma asíncrona.

La ejecución de los trabajos requiere un proceso worker de Laravel configurado en el entorno de despliegue cuando la aplicación utilice trabajos en cola.

4.9. Gestión de sesiones y almacenamiento

Las sesiones de usuario utilizan el almacenamiento basado en archivos:

SESSION_DRIVER=file

El almacenamiento de archivos de Laravel está configurado mediante:

FILESYSTEM_DISK=local

La aplicación se ejecuta en un contenedor Docker en Render y actualmente no dispone de almacenamiento persistente asociado.

Por este motivo, la arquitectura no debe depender del almacenamiento local del contenedor para conservar información que deba persistir entre despliegues o reemplazos de la instancia.

4.10. Despliegue mediante Docker

La aplicación se empaqueta mediante Docker y se despliega en Render.

El código fuente se encuentra alojado en GitHub y está conectado con Render, permitiendo automatizar la construcción y el despliegue de la aplicación.

La utilización de Docker permite definir de forma reproducible el entorno necesario para ejecutar la aplicación y sus dependencias.

4.11. Envío de correo electrónico

La aplicación utiliza el sistema de correo de Laravel y define los mensajes mediante clases Mailable.

Actualmente el transporte está configurado mediante:

```
MAIL_MAILER=log
```

Por tanto, los mensajes generados por la aplicación se registran actualmente en los logs en lugar de enviarse mediante un servicio de correo externo.

La configuración de un proveedor de correo externo en el futuro constituiría una nueva dependencia de la aplicación.

4.12. Resumen de la estrategia

La solución se basa en una aplicación web monolítica con arquitectura MVC, implementada mediante Laravel y complementada con mecanismos específicos para separar responsabilidades.

Las principales decisiones arquitectónicas son:

1. Utilización de Laravel como framework principal.
2. Arquitectura MVC.
3. Separación de responsabilidades mediante Services, Requests, Policies, Rules, Commands, Middleware, Exceptions, Enums, Query Filters, Helpers y Providers.
4. Uso de Blade y Livewire para la presentación.
5. Uso de PostgreSQL y Eloquent para la persistencia.
6. Externalización de PostgreSQL mediante Neon.
7. Utilización de PostgreSQL para caché y almacenamiento de trabajos en cola.
8. Integración con Stripe para los pagos.
9. Contenerización mediante Docker.
10. Despliegue en Render.
11. Integración de GitHub con Render para el despliegue automático.

5. Vista de bloques de construcción

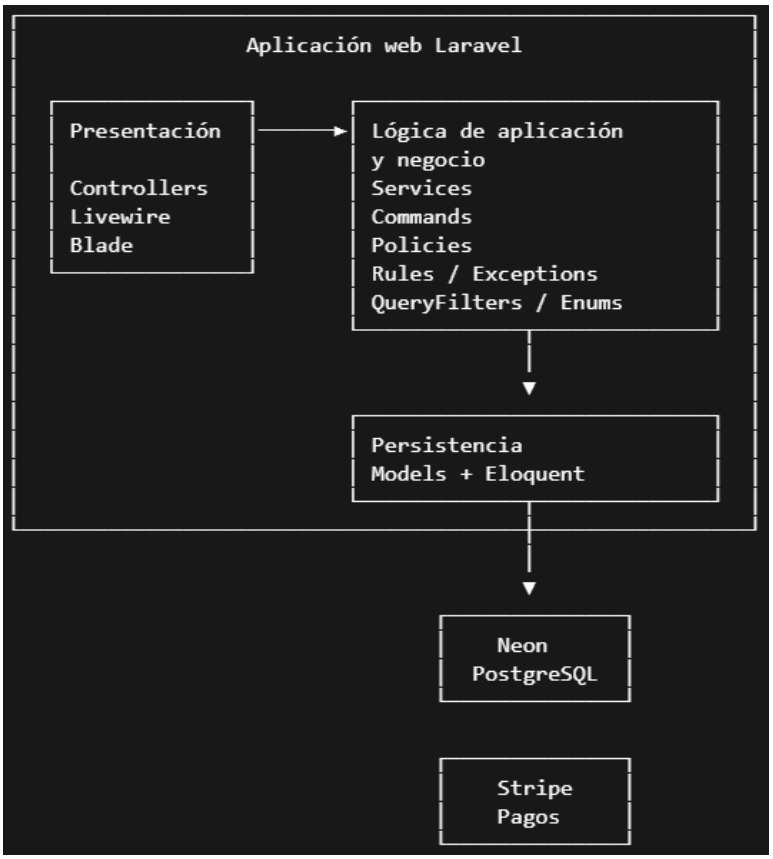
La vista de bloques de construcción describe la estructura estática de la aplicación y la distribución de sus principales responsabilidades.

La aplicación está implementada como un monolito Laravel basado en el patrón MVC. Dentro de esta arquitectura, los componentes se organizan en diferentes bloques que permiten separar las responsabilidades de presentación, lógica de negocio, persistencia, validación, autorización e integración con servicios externos.

La organización de los componentes sigue en determinados casos una separación por ámbito funcional, principalmente para las operaciones correspondientes a administradores, clientes, gestores y funcionalidades comunes.

5.1. Nivel 1: Sistema

En el nivel más alto, el sistema está compuesto por una única aplicación web monolítica desarrollada con Laravel.



5.2. Nivel 2: Bloques principales

La aplicación se divide en los siguientes bloques principales:

Bloque	Responsabilidad principal
Presentación	Interacción con el usuario y gestión de peticiones HTTP
Aplicación y lógica de negocio	Implementación de operaciones y reglas de negocio
Persistencia	Representación y acceso a los datos
Validación y autorización	Validación de entradas y control de permisos
Integraciones externas	Comunicación con servicios externos como Stripe
Infraestructura y componentes transversales	Configuración y funcionalidades reutilizables

Estos bloques no constituyen procesos o servicios independientes, sino agrupaciones lógicas de componentes dentro de la misma aplicación monolítica.

5.3. Presentación y entrada al sistema

Este bloque contiene los componentes que actúan como punto de entrada de las interacciones con los usuarios y permiten generar la interfaz web.

5.3.1. Controllers

Los controladores se encuentran en app/Http/Controllers.

Se organizan por ámbito funcional. Esta organización permite separar las operaciones según el perfil o contexto al que pertenecen.

1. Public. Contiene las funcionalidades accesibles sin necesidad de autenticación:
    1. HomeController. Permite gestionar las operaciones relacionadas con la parte pública de la aplicación.
2. Common. Contiene controladores utilizados por diferentes perfiles:
    1. ProfileController. Gestiona las operaciones relacionadas con el perfil del usuario.
    2. StripeWebhookController. Recibe y valida los eventos enviados por Stripe y delega su procesamiento en StripeWebhookService.

3. Client. Contiene los controladores correspondientes a las operaciones realizadas por los clientes:
    1. DashboardController. Gestiona las operaciones del panel de control.
    2. FeedbackController. Gestiona el historial y envío de comentarios.
    3. ReservationController. Gestiona todas las operaciones relacionadas con las reservas de los clientes.
4. Manager. Contiene los controladores correspondientes a la gestión de la aplicación:
    1. CourtController
    2. DashboardController
    3. FeedbackController
    4. ReservationController

    Estos controladores permiten gestionar pistas, reservas y comentarios desde el área de gestión.
5. Admin. Contiene los controladores relacionados con las operaciones exclusivas del administrador:
    1. ManagerController. Se encarga de gestionar las operaciones relacionadas con la administración de los gestores del sistema.
6. Auth. Contiene los controladores relacionados con determinadas operaciones de autenticación:
    1. VerifyEmailController. Gestiona la validación del email del cliente.

5.3.2. Livewire

Los componentes Livewire se encuentran en app/Livewire.

Estos componentes permiten implementar determinadas interacciones dinámicas de la interfaz de usuario sin separar el frontend en una aplicación independiente.

5.3.3. Blade y componentes de vista

Las vistas Blade constituyen la capa de presentación de la aplicación.

Los componentes reutilizables de la interfaz se encuentran en app/View/Components.

Estos componentes permiten reutilizar elementos comunes de la interfaz y mantener una estructura consistente entre las diferentes vistas.

5.4. Lógica de aplicación y negocio

La lógica de aplicación se encuentra principalmente en app/Services.

Los servicios están organizados por ámbito funcional. Esta organización permite mantener separadas las operaciones correspondientes a cada contexto funcional.

1. Services de administración. En Services/Admin se encuentran:
    1. ManagerCreateService
    2. ManagerUpdateService

    Estos servicios encapsulan las operaciones de creación y modificación de gestores, respectivamente.
2. Services de cliente. En Services/Client se encuentra:
    1. ReservationCheckoutService. Encapsula las operaciones necesarias para realizar el proceso de reserva y checkout por parte de un cliente.
3. Services comunes. En Services/Common se encuentran:
    1. CourtScheduleService
    2. ProfileService
    3. ReservationCancellationService
    4. StripeWebhookService

    Estos servicios contienen operaciones utilizadas por diferentes partes de la aplicación. Entre ellas se encuentran la gestión de horarios de pistas, la gestión del perfil, la cancelación de reservas y el procesamiento de eventos recibidos desde Stripe.
4. Services de gestor. En Services/Manager se encuentran:
    1. QuickReservationService
    2. ReservationCreateService
    3. ReservationRescheduleService
    4. ReservationUpdateService

Estos servicios encapsulan las operaciones de gestión de reservas realizadas desde el área de gestores.

5. Services de validación. En Services/Validation se encuentran:
    1. ReservationRulesValidator
    2. ReservationRuleResult

    Estos componentes centralizan la validación de las condiciones necesarias para determinadas operaciones relacionadas con las reservas.

5.5. Persistencia

La persistencia se encuentra principalmente en app/Models.

Actualmente se identifican los siguientes modelos principales:

1. User
2. Court
3. Reservation
4. Feedback

Estos modelos representan las principales entidades persistentes de la aplicación.

Los modelos utilizan Eloquent ORM para acceder a PostgreSQL.

5.5.1. Concerns de los modelos

Los modelos incluyen determinados comportamientos reutilizables mediante Concerns:

1. Filterable
2. Sortable

Estos componentes proporcionan funcionalidades reutilizables relacionadas con el filtrado y ordenación de consultas, respectivamente.

5.6. Validación y autorización

La validación y autorización se distribuyen entre diferentes componentes especializados.

5.6.1. Form Requests

Los Form Requests se encuentran en app/Http/Requests y están organizados según el ámbito funcional.

Estos componentes encapsulan la validación de los datos recibidos mediante las peticiones HTTP.

Entre las operaciones validadas se encuentran la creación y modificación de gestores, creación y modificación de reservas, cancelación y reprogramación de reservas, gestión de pistas y envío de comentarios.

5.6.2. Rules

Las reglas de validación específicas se encuentran en app/Rules:

1. ValidDni. Encapsula la validación específica del DNI.

5.6.3. Policies

Las reglas de autorización se encuentran en app/Policies:

1. ReservationPolicy. Determina las operaciones que pueden realizar los usuarios sobre las reservas según las reglas de autorización definidas.

5.6.4. Middleware

La aplicación dispone de un middleware específico:

1. RoleMiddleware. Permite controlar el acceso a determinadas partes de la aplicación en función del rol del usuario.

   De esta forma, la autorización se implementa mediante diferentes mecanismos complementarios.

Cada mecanismo tiene una responsabilidad diferente dentro del proceso de autorización y validación.

5.7. Filtrado y consulta de datos

Los filtros de consulta se encuentran en app/QueryFilters:

1. QueryFilter
2. CourtFilter
3. FeedbackFilter
4. ReservationFilter

5. UserFilter

Estos componentes permiten encapsular criterios de filtrado y reutilizarlos en las consultas correspondientes.

La utilización de Query Filters evita concentrar la lógica de filtrado directamente en los controladores y facilita la composición de consultas.

5.8. Integración con Stripe

La integración con Stripe está encapsulada principalmente mediante:

1. StripeWebhookController
2. StripeWebhookService

El controlador recibe el contenido del webhook y la cabecera Stripe-Signature.

La firma se verifica utilizando el secreto configurado para los webhooks de Stripe.

Una vez validado el evento, el procesamiento se delega en StripeWebhookService.

De esta forma, el controlador se mantiene centrado en la recepción y validación inicial de la petición, mientras que el servicio encapsula el procesamiento del evento.

5.9. Comandos

Los comandos personalizados de Laravel se encuentran en app/Console/Commands:

1. CancelExpiredReservations. Se encarga de realizar las operaciones relacionadas con la cancelación de reservas expiradas.

Los comandos permiten ejecutar operaciones de mantenimiento o procesamiento que no dependen directamente de una petición HTTP de un usuario.

5.10. Enums

Los valores pertenecientes a conjuntos cerrados se representan mediante enumeraciones ubicadas en app/Enums:

1. UserRole. Representa los roles de usuario.
2. PaymentStatus. Representa los estados relacionados con los pagos.
3. CanceledBy. Representa quién realiza la cancelación de una reserva.
4. FeedbackType. Representa los tipos de comentarios.

La utilización de Enums evita depender de valores literales dispersos por el código y proporciona una representación explícita de estos conceptos.

5.11. Excepciones

Las excepciones específicas de la aplicación se encuentran en app/Exceptions:

1. PastDateException
2. RefundException
3. ReservationNotCancellableException
4. ReservationNotResumableException
5. SlotUnavailableException

Estas excepciones permiten representar explícitamente diferentes situaciones de negocio relacionadas principalmente con la gestión de reservas y pagos.

5.12. Helpers

La aplicación dispone de un helper en app/Helpers:

1. SortHelper. Este componente proporciona funcionalidad auxiliar reutilizable relacionada con la ordenación.

Los helpers contienen funcionalidades comunes que pueden ser utilizadas por diferentes componentes de la aplicación.

5.13. Providers

Los Service Providers se encuentran en app/Providers:

1. AppServiceProvider
2. VoltServiceProvider

Los Providers permiten registrar y configurar servicios y componentes durante el arranque de la aplicación.

5.14. Correo electrónico

Las clases encargadas de representar los correos electrónicos se encuentran en app/Mail:

1. ReservationCanceledByManagerMail
2. ReservationRescheduledMail

Estas clases extienden del mecanismo Mailable de Laravel y representan los mensajes que puede generar la aplicación.

Actualmente el transporte de correo está configurado mediante MAIL_MAILER=log, por lo que los mensajes se registran en los logs de la aplicación en lugar de enviarse mediante un proveedor externo.

6. Vista de ejecución

La vista de ejecución describe cómo interactúan los principales componentes de la aplicación durante la ejecución de determinados casos de uso.

Debido a que la aplicación es un proyecto destinado principalmente a portfolio, se han seleccionado únicamente los escenarios más representativos de la arquitectura, evitando documentar individualmente todas las operaciones disponibles.

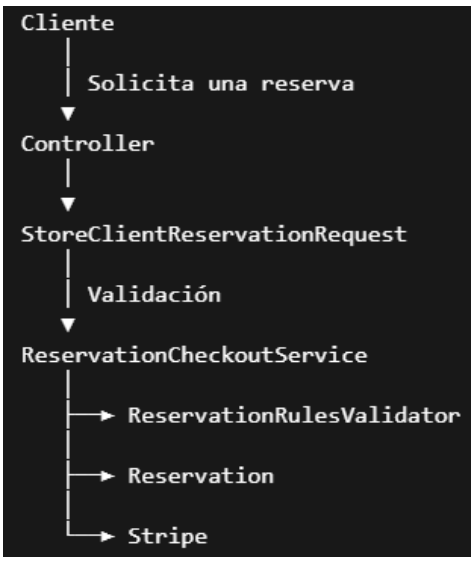
Los escenarios seleccionados son:

1. Realización de una reserva.
2. Recepción de un webhook de Stripe.
3. Cancelación de una reserva.
4. Cancelación automática de reservas expiradas.

6.1. Realización de una reserva

El proceso de realización de una reserva por parte de un cliente involucra los componentes encargados de recibir la petición, validar los datos, aplicar las reglas de negocio, gestionar la reserva y realizar el proceso de pago.

De forma simplificada, el flujo es:



El cliente selecciona una pista y una franja horaria y solicita realizar la reserva.

La petición es recibida por el controlador correspondiente y los datos de entrada se validan mediante StoreClientReservationRequest.

A continuación, la operación se delega en ReservationCheckoutService, que coordina el proceso de reserva y pago.

Durante este proceso se comprueban las condiciones necesarias para determinar si la reserva puede realizarse mediante ReservationRulesValidator.

La aplicación utiliza Stripe como pasarela de pago.

Este flujo permite mantener en el controlador únicamente la gestión de la petición HTTP, delegando el procesamiento de la operación en un servicio específico.

6.2. Recepción de un webhook de Stripe

Stripe puede enviar notificaciones a la aplicación mediante webhooks. La aplicación dispone de un controlador y un servicio específicos para gestionar estos eventos.

StripeWebhookController recibe la petición enviada por Stripe y obtiene el contenido del webhook y la cabecera Stripe-Signature.

La firma se verifica utilizando el secreto configurado para los webhooks de Stripe.

Si la firma no es válida, la aplicación rechaza la petición y devuelve una respuesta HTTP 400.

Si la firma es válida, el controlador delega el procesamiento del evento en StripeWebhookService.

De esta forma, la recepción y validación de la comunicación HTTP se mantiene separada del procesamiento de los eventos recibidos.

6.3. Cancelación de una reserva

Los clientes pueden cancelar sus reservas cuando se cumplen las condiciones establecidas por la aplicación.

La petición de cancelación es recibida por el controlador correspondiente.

La aplicación comprueba que la operación está autorizada mediante ReservationPolicy.

Posteriormente, ReservationCancellationService se encarga de procesar la cancelación y realizar las comprobaciones necesarias.

Si la operación no puede realizarse, la aplicación dispone de excepciones específicas como ReservationNotCancellableException para representar esta situación.

Cuando la operación es válida, se actualiza la información correspondiente de la reserva y los cambios se persisten mediante Eloquent en PostgreSQL.

6.4. Cancelación automática de reservas expiradas

La aplicación dispone del comando CancelExpiredReservations, destinado a procesar las reservas que han expirado.

El comando se ejecuta independientemente de una petición HTTP realizada por un usuario.

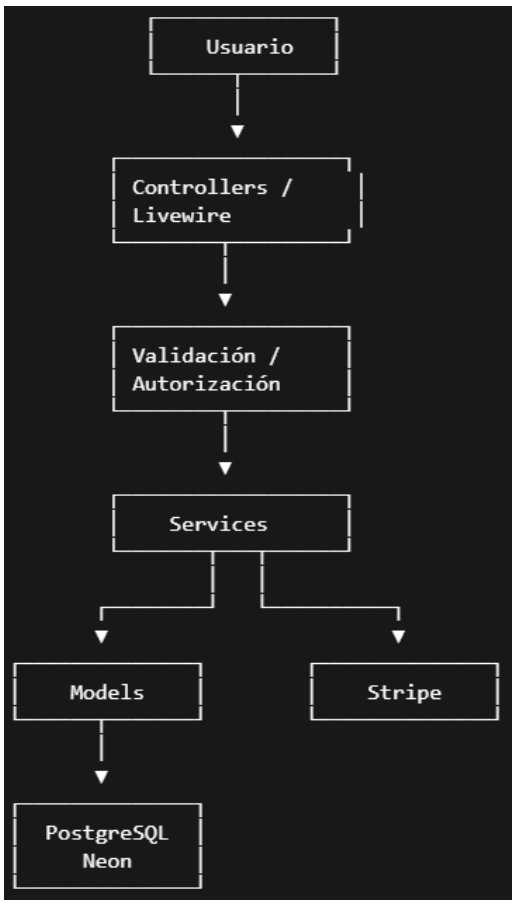
Durante su ejecución se consultan las reservas y se identifican aquellas que cumplen las condiciones establecidas para considerarse expiradas.

Las reservas correspondientes se actualizan y los cambios se almacenan en PostgreSQL.

Este mecanismo permite realizar tareas de mantenimiento de forma independiente de la interfaz web.

6.5. Resumen de los flujos de ejecución

Los escenarios anteriores muestran el patrón general de funcionamiento de la aplicación:



La aplicación sigue un flujo general en el que los componentes de presentación reciben las peticiones y delegan las operaciones en componentes especializados.

Los mecanismos de validación y autorización se aplican según las necesidades de cada operación. Los Services coordinan las operaciones de negocio y utilizan los Models para acceder a los datos persistentes.

Las comunicaciones con servicios externos, como Stripe, se realizan mediante componentes específicos de integración.

Esta organización permite mantener separadas las principales responsabilidades de la aplicación y facilita la comprensión de los flujos de ejecución sin necesidad de detallar individualmente todas las funcionalidades del sistema.

7. Vista de despliegue

La vista de despliegue describe la infraestructura sobre la que se ejecuta la aplicación y las relaciones entre los diferentes elementos que participan en su ejecución.

La aplicación se despliega como un servicio web Dockerizado en Render. El código fuente se encuentra alojado en GitHub y Render está conectado con el repositorio para realizar los despliegues de la aplicación.

La base de datos PostgreSQL se encuentra externalizada en Neon y los pagos se gestionan mediante Stripe.

7.1. Infraestructura general

La aplicación se ejecuta en un Web Service de Render mediante un contenedor Docker.

La comunicación con PostgreSQL y Stripe se realiza desde la aplicación desplegada en Render.

7.2. Código fuente y proceso de despliegue

El código fuente de la aplicación se encuentra alojado en GitHub.

Render está conectado al repositorio y configurado para realizar un nuevo despliegue cuando se realizan cambios en la rama configurada.

El servicio utiliza el siguiente Dockerfile:

docker/php/Dockerfile.render

El contexto de construcción de Docker corresponde al directorio raíz del proyecto.

La configuración de Render utiliza el despliegue automático ante nuevos commits.

7.3. Contenedor de aplicación

La aplicación se ejecuta dentro de un contenedor Docker.

El contenedor proporciona el entorno necesario para ejecutar la aplicación Laravel y sus dependencias.

Los principales componentes de ejecución son:

1. Nginx como servidor web.
2. PHP 8.4 como entorno de ejecución.
3. Laravel 12.62.0 como framework de aplicación.
4. Vite para la compilación y gestión de recursos frontend.

El uso de Docker permite definir el entorno de ejecución de la aplicación de forma reproducible y desacoplada del entorno local de desarrollo.

7.4. Servidor web

Nginx actúa como servidor web y recibe las peticiones HTTP dirigidas a la aplicación.

Las peticiones son procesadas por la aplicación Laravel mediante PHP.

7.5. Base de datos

La aplicación utiliza PostgreSQL como sistema gestor de base de datos.

La base de datos no se encuentra dentro del contenedor de Render, sino que está externalizada en Neon.

La comunicación se realiza mediante la conexión configurada en Laravel.

La externalización de la base de datos permite mantener los datos persistentes fuera del ciclo de vida del contenedor de la aplicación.

7.6. Persistencia del almacenamiento local

La aplicación utiliza actualmente:

FILESYSTEM_DISK=local

Sin embargo, el servicio de Render no tiene configurado un disco persistente. Por tanto, el almacenamiento local del contenedor no debe considerarse un mecanismo de persistencia permanente para información que deba conservarse entre diferentes instancias o despliegues.

La información persistente principal de la aplicación se almacena en PostgreSQL.

Esta consideración es especialmente relevante para cualquier funcionalidad futura que requiera almacenar archivos.

7.7. Sesiones, caché y colas

La aplicación utiliza diferentes mecanismos de almacenamiento proporcionados por Laravel.

Las sesiones están configuradas mediante:

SESSION_DRIVER=file

Por tanto, las sesiones se almacenan en el sistema de archivos del entorno de ejecución.

La caché utiliza:

CACHE_STORE=database

Por lo que los datos de caché se almacenan en PostgreSQL.

Las colas utilizan:

QUEUE_CONNECTION=database

Por lo que los trabajos pendientes se almacenan también en PostgreSQL.

En consecuencia, la arquitectura evita actualmente la necesidad de incorporar un servicio externo como Redis para estos mecanismos.

7.8. Servicio de correo

La aplicación utiliza las clases Mailable proporcionadas por Laravel para representar los mensajes de correo.

Actualmente el transporte está configurado como:

MAIL_MAILER=log

Por tanto, durante el despliegue actual los mensajes no se envían mediante un proveedor de correo externo, sino que se registran en los logs de la aplicación.

La configuración de un proveedor externo de correo sería necesaria si la aplicación pasara a utilizarse en un entorno real.

7.9. Integración con Stripe

Stripe constituye un servicio externo utilizado por la aplicación para gestionar los pagos.

La aplicación se comunica con Stripe desde el contenedor desplegado en Render.

Además, Stripe puede comunicarse con la aplicación mediante webhooks. Los webhooks permiten que Stripe notifique a la aplicación determinados eventos relacionados con los pagos.

7.10. Entorno de despliegue

El servicio web se encuentra desplegado en Render, en la región Frankfurt (EU Central).

La aplicación está disponible mediante el dominio proporcionado por Render.

7.11. Resumen

La aplicación utiliza una arquitectura de despliegue sencilla, basada en un único Web Service Dockerizado en Render.

El código fuente se gestiona mediante GitHub y los cambios en el repositorio desencadenan automáticamente nuevos despliegues.

La aplicación se ejecuta mediante Nginx y PHP 8.4 dentro del entorno proporcionado por Docker.

Los datos persistentes se almacenan en una base de datos PostgreSQL externa proporcionada por Neon, mientras que Stripe se utiliza como servicio externo para la gestión de pagos.

Esta configuración permite separar el ciclo de vida del contenedor de la persistencia principal de los datos y mantiene una infraestructura relativamente sencilla para las necesidades del proyecto.

8. Conceptos transversales

Los conceptos transversales son aquellos aspectos de la aplicación que afectan a diferentes componentes y funcionalidades del sistema.

En este proyecto se han identificado los siguientes conceptos transversales:

1. Autenticación y autorización.
2. Validación de datos.
3. Gestión de errores y excepciones.
4. Persistencia de datos.
5. Gestión de reservas.
6. Integración con servicios externos.
7. Envío de correos.
8. Gestión de sesiones, caché y colas.

8.1. Autenticación y autorización

La autenticación de los usuarios se realiza mediante los mecanismos proporcionados por Laravel.

El sistema distingue tres perfiles de usuario:

1. Cliente.
2. Gestor.
3. Administrador.

La autorización se realiza mediante diferentes mecanismos dependiendo de la operación.

Para las operaciones relacionadas con reservas se utiliza ReservationPolicy, mientras que el acceso a las diferentes áreas de la aplicación se controla mediante middleware.

La aplicación dispone de RoleMiddleware, encargado de comprobar el rol del usuario antes de permitir el acceso a determinadas funcionalidades.

Los permisos relacionados con las reservas se gestionan mediante ReservationPolicy.

De esta forma, la autenticación determina quién es el usuario y los mecanismos de autorización determinan qué operaciones puede realizar.

8.2. Validación de datos

La validación de los datos recibidos desde las peticiones HTTP se realiza principalmente mediante las clases Form Request de Laravel.

Los Form Request permiten centralizar las reglas de validación de las diferentes operaciones.

Además de las reglas proporcionadas por Laravel, la aplicación dispone de reglas personalizadas, como ValidDni, para aquellas validaciones específicas del dominio.

La validación de las reservas se complementa con ReservationRulesValidator, que permite comprobar condiciones relacionadas con la lógica de negocio.

De esta forma se diferencia entre:

1. Validación de los datos recibidos.
2. Comprobación de las reglas específicas del dominio.

8.3. Gestión de errores y excepciones

La aplicación utiliza excepciones específicas para representar determinadas situaciones que impiden completar una operación.

Entre las excepciones definidas se encuentran:

1. PastDateException.
2. RefundException.
3. ReservationNotCancellableException.
4. ReservationNotResumableException.
5. SlotUnavailableException.

Estas excepciones permiten representar situaciones concretas de la lógica de negocio en lugar de utilizar únicamente excepciones genéricas.

Por ejemplo, una reserva que no puede cancelarse puede provocar ReservationNotCancellableException, mientras que la disponibilidad de una franja horaria puede provocar SlotUnavailableException.

Este enfoque permite separar las condiciones excepcionales de la lógica normal de ejecución.

8.4. Persistencia de datos

La aplicación utiliza PostgreSQL como sistema gestor de base de datos y Eloquent como ORM proporcionado por Laravel.

Los principales modelos de la aplicación son:

1. User.
2. Court.
3. Reservation.
4. Feedback.

Los modelos representan las principales entidades persistentes de la aplicación y se encargan de interactuar con la base de datos mediante Eloquent.

La base de datos PostgreSQL se encuentra externalizada en Neon.

La persistencia se utiliza para almacenar, entre otros datos:

1. Usuarios.
2. Pistas deportivas.
3. Reservas.

4. Comentarios.
5. Información relacionada con los pagos.
6. Información necesaria para sesiones, caché y colas.

8.5. Gestión de reservas

Las reservas constituyen una de las principales entidades del sistema y su gestión está distribuida entre diferentes componentes.

La lógica relacionada con las operaciones de reservas se encuentra principalmente encapsulada en Services, entre los que se encuentran:

1. ReservationCheckoutService.
2. ReservationCancellationService.
3. ReservationCreateService.
4. ReservationRescheduleService.
5. ReservationUpdateService.
6. QuickReservationService.

La validación de las condiciones necesarias para realizar determinadas operaciones se centraliza parcialmente en ReservationRulesValidator.

Además, se utilizan excepciones específicas para representar situaciones en las que una operación no puede realizarse.

Esta organización evita concentrar toda la lógica de las reservas directamente en los controladores.

Las operaciones de consulta que requieren filtrado utilizan las clases situadas en app/QueryFilters.

Actualmente existen filtros para diferentes entidades, entre ellas:

1. Usuarios.
2. Reservas.
3. Pistas.
4. Comentarios.

Los modelos también utilizan funcionalidades comunes relacionadas con el filtrado y la ordenación mediante los concerns Filterable y Sortable.

Este mecanismo permite reutilizar la lógica de consulta y mantener los controladores más sencillos.

8.6. Integración con Stripe

Stripe se utiliza como pasarela de pago.

La integración se encuentra encapsulada en componentes específicos de la aplicación, principalmente:

1. ReservationCheckoutService.
2. StripeWebhookController.
3. StripeWebhookService.

El controlador de webhook se encarga de recibir las notificaciones de Stripe y verificar su firma.

Una vez validado el evento, su procesamiento se delega en StripeWebhookService.

De esta forma, los detalles relacionados con la comunicación con Stripe no se encuentran distribuidos por toda la aplicación.

8.7. Envío de correos

Los mensajes de correo se implementan mediante las clases Mailable proporcionadas por Laravel.

Actualmente existen las clases:

1. ReservationCanceledByManagerMail.
2. ReservationRescheduledMail.

Estas clases representan los mensajes que deben enviarse en determinadas situaciones relacionadas con las reservas.

Actualmente el proyecto utiliza:

MAIL_MAILER=log

Por lo que los mensajes se registran en los logs en lugar de enviarse mediante un proveedor externo de correo.

La utilización de Mailable permite mantener separada la definición del contenido de los mensajes de la lógica que provoca su generación.

8.8. Sesiones, caché y colas

La aplicación utiliza mecanismos proporcionados por Laravel para gestionar sesiones, caché y trabajos en cola.

La configuración actual es:

SESSION_DRIVER=file

CACHE_STORE=database

QUEUE_CONNECTION=database

Las sesiones utilizan el sistema de archivos del entorno de ejecución.

La caché utiliza PostgreSQL mediante el driver database.

Las colas utilizan igualmente PostgreSQL mediante el driver database.

No se utiliza actualmente un servicio externo como Redis para estos mecanismos.

8.9. Interfaz de usuario

La interfaz utiliza Blade como sistema de plantillas y Livewire para implementar componentes interactivos.

Vite se utiliza para la gestión y compilación de los recursos frontend.

La aplicación combina, por tanto, renderizado del lado del servidor mediante Blade con componentes interactivos proporcionados por Livewire.

Esta combinación permite implementar interacciones dinámicas sin necesidad de desarrollar una aplicación frontend independiente basada en un framework JavaScript completo.

8.10. Organización de responsabilidades

La lógica de las operaciones más complejas se encuentra principalmente en la capa de Services.

Los servicios están organizados por áreas funcionales.

Esta organización permite que los controladores actúen principalmente como puntos de entrada de las peticiones y deleguen las operaciones de mayor complejidad en servicios especializados.

Por ejemplo, las operaciones relacionadas con las reservas se distribuyen entre diferentes servicios especializados, como:

1. ReservationCheckoutService
2. ReservationCancellationService
3. ReservationCreateService
4. ReservationRescheduleService
5. ReservationUpdateService

De esta forma, la lógica relacionada con una operación concreta no queda concentrada en un único controlador.

8.11. Presentación e interacción con el usuario

La interfaz de usuario utiliza Blade como sistema de plantillas y Livewire para proporcionar componentes interactivos.

Vite se utiliza para gestionar los recursos frontend.

La aplicación combina, por tanto, renderizado mediante Blade con componentes Livewire cuando se requiere una mayor interacción con el usuario sin implementar una aplicación frontend independiente.

La lógica de presentación se mantiene separada de los modelos y de los servicios que implementan las operaciones de aplicación.

9. Decisiones arquitectónicas

En esta sección se recogen las principales decisiones arquitectónicas adoptadas durante el desarrollo de la aplicación y los motivos que llevaron a su elección.

No se incluyen decisiones relacionadas exclusivamente con detalles de implementación, sino aquellas que tienen una influencia significativa sobre la estructura, el funcionamiento o el mantenimiento de la aplicación.

9.1. Utilización de Laravel

Se ha seleccionado Laravel como framework principal para el desarrollo de la aplicación. Laravel proporciona una estructura de aplicación web completa y mecanismos integrados para aspectos habituales del desarrollo web, como: enrutamiento, autenticación, autorización, etc.

La utilización de un framework completo permite evitar implementar desde cero funcionalidades comunes y proporciona una estructura organizada sobre la que desarrollar la aplicación.

Además, Laravel permite integrar las diferentes tecnologías utilizadas en el proyecto manteniendo una estructura coherente.

9.2. Utilización de una arquitectura MVC

La aplicación utiliza el patrón arquitectónico Modelo-Vista-Controlador (MVC) proporcionado por Laravel.

MVC permite separar las principales responsabilidades de la aplicación:

1. Modelos. Representan los datos y su interacción con la base de datos.
2. Vistas. Representan la interfaz que recibe el usuario.
3. Controladores. Gestionan las peticiones y coordinan las operaciones correspondientes.

Esta separación proporciona una estructura clara para una aplicación web y facilita la organización del código.

Sin embargo, debido a que algunas operaciones de la aplicación contienen lógica más compleja, la lógica de aplicación no se concentra exclusivamente en los controladores.

9.3. Utilización de Services

Se ha decidido utilizar una capa de Services para encapsular operaciones de aplicación y lógica que requiere coordinación entre diferentes componentes.

Esta decisión evita concentrar operaciones complejas directamente en los controladores.

Por ejemplo, las operaciones relacionadas con las reservas se distribuyen entre servicios específicos como:

1. ReservationCheckoutService
2. ReservationCancellationService
3. ReservationCreateService
4. ReservationRescheduleService
5. ReservationUpdateService

De esta manera, los controladores actúan principalmente como puntos de entrada y delegan el procesamiento en componentes especializados.

Esta organización también permite reutilizar determinadas operaciones desde diferentes puntos de entrada de la aplicación.

9.4. Utilización de PostgreSQL

Se ha seleccionado PostgreSQL como sistema gestor de base de datos.

La aplicación trabaja principalmente con datos estructurados y relacionados, como usuarios, pistas, reservas y comentarios.

PostgreSQL proporciona:

1. Modelo relacional.
2. Integridad referencial.
3. Transacciones.
4. Soporte adecuado para las relaciones utilizadas por la aplicación.
5. Integración con Laravel mediante Eloquent ORM.

El uso de una base de datos relacional resulta adecuado para el modelo de datos de la aplicación, especialmente para las relaciones existentes entre usuarios, reservas y pistas.

9.5. Externalización de la base de datos en Neon

La base de datos PostgreSQL se mantiene fuera del entorno de ejecución de la aplicación y se utiliza Neon como proveedor.

El contenedor Docker que ejecuta la aplicación puede ser reemplazado durante los despliegues. Por este motivo, los datos persistentes no deben depender del almacenamiento local del contenedor.

9.6. Utilización de Docker para el despliegue

La aplicación se ejecuta mediante Docker y se despliega en Render utilizando una imagen construida a partir de un Dockerfile específico.

Docker permite definir el entorno de ejecución de la aplicación de forma reproducible.

El contenedor incluye las dependencias necesarias para ejecutar la aplicación, evitando depender de la configuración específica del sistema operativo donde se ejecute.

Además, la utilización de Docker facilita trasladar el mismo entorno de ejecución entre desarrollo y despliegue con las configuraciones necesarias.

9.7. Utilización de Render como plataforma de despliegue

Se ha seleccionado Render como plataforma para desplegar la aplicación.

Render permite desplegar directamente la aplicación desde el repositorio de GitHub y automatizar el proceso de construcción y despliegue. Así se evita tener que administrar directamente un servidor propio.

9.8. Utilización de Blade y Livewire

Se ha optado por utilizar Blade y Livewire para la interfaz de usuario.

Blade proporciona el sistema de plantillas necesario para generar las páginas de la aplicación, mientras que Livewire permite implementar componentes interactivos manteniendo gran parte de la lógica dentro del ecosistema Laravel.

Esta decisión reduce la complejidad de la aplicación al evitar mantener dos aplicaciones independientes, una para el backend y otra para el frontend.

Vite se utiliza para gestionar y compilar los recursos frontend.

9.9. Utilización de Stripe para los pagos

Se ha seleccionado Stripe como pasarela de pago.

La gestión de pagos requiere interactuar con un proveedor especializado y no resulta apropiado implementar directamente el procesamiento de pagos dentro de la aplicación.

Stripe proporciona la infraestructura necesaria para gestionar el proceso de pago y permite comunicar determinados eventos a la aplicación mediante webhooks.

La integración se ha aislado mediante componentes específicos, como StripeWebhookController y StripeWebhookService.

De esta manera, la aplicación mantiene separada la lógica específica de Stripe del resto de la lógica de aplicación.

9.10. Separación de validación y autorización

Se ha decidido utilizar mecanismos específicos de Laravel para separar la validación de datos y la autorización de operaciones.

La validación de las peticiones se realiza principalmente mediante Form Requests, mientras que la autorización se gestiona mediante Middleware y Policies.

Esta separación permite evitar que los controladores tengan que contener directamente todas las comprobaciones relacionadas con los datos recibidos y los permisos de los usuarios.

Esta organización mejora la separación de responsabilidades entre los diferentes componentes.

9.11. Uso de mecanismos de Laravel para tareas transversales

Siempre que ha sido posible, se han utilizado los mecanismos proporcionados por Laravel para resolver funcionalidades comunes.

Utilizar mecanismos integrados en el framework permite mantener una arquitectura coherente y reducir la cantidad de infraestructura desarrollada específicamente para el proyecto.

Además, estos mecanismos siguen las convenciones habituales del framework, lo que facilita la comprensión del proyecto para desarrolladores familiarizados con Laravel.

10. Requisitos de calidad

Los requisitos de calidad describen los principales atributos de calidad considerados relevantes para la aplicación y cómo la arquitectura contribuye a satisfacerlos.

Debido a que la aplicación ha sido desarrollada como proyecto de portfolio y no está destinada actualmente a un entorno de producción con una carga de usuarios determinada, no se establecen objetivos cuantitativos de rendimiento o disponibilidad.

Se consideran principalmente los siguientes atributos de calidad:

1. Mantenibilidad.
2. Seguridad.
3. Fiabilidad.
4. Rendimiento.
5. Usabilidad.

6. Desplegabilidad.

10.1. Mantenibilidad

Escenario:

Un desarrollador necesita modificar o ampliar una funcionalidad existente, por ejemplo, el proceso de gestión de reservas.

Respuesta arquitectónica:

La aplicación separa las diferentes responsabilidades mediante Controllers, Services, Models, Form Requests, Policies, Query Filters y otros componentes especializados.

Las operaciones de mayor complejidad se encuentran encapsuladas en Services específicos.

Por ejemplo, las operaciones relacionadas con las reservas se distribuyen entre diferentes servicios:

1. ReservationCheckoutService
2. ReservationCancellationService
3. ReservationCreateService
4. ReservationRescheduleService
5. ReservationUpdateService

Esta separación permite modificar una parte de la lógica sin tener que concentrar todos los cambios en un único componente.

La utilización de una estructura convencional de Laravel también facilita que un desarrollador familiarizado con el framework pueda localizar los diferentes elementos de la aplicación.

10.2. Seguridad

Escenario:

Un usuario intenta acceder a una funcionalidad o realizar una operación que no corresponde a sus permisos.

Respuesta arquitectónica:

La aplicación utiliza diferentes mecanismos para controlar el acceso:

1. Autenticación de Laravel.
2. Middleware para controlar el acceso según el rol.
3. Policies para controlar operaciones sobre determinados recursos.
4. Form Requests para validar los datos recibidos.

Los roles se representan mediante el enum UserRole.

Además, la integración con Stripe mediante webhooks incluye la verificación de la firma proporcionada por Stripe antes de procesar el evento.

Esto evita que una petición externa pueda ser tratada como un webhook válido sin superar previamente la comprobación de autenticidad correspondiente.

10.3. Fiabilidad

Escenario:

Una operación relacionada con una reserva no puede realizarse porque no se cumplen las condiciones necesarias.

Respuesta arquitectónica:

La aplicación utiliza reglas de negocio específicas y excepciones personalizadas para representar situaciones que impiden completar determinadas operaciones.

Entre ellas se encuentran:

1. SlotUnavailableException
2. ReservationNotCancellableException
3. ReservationNotResumableException
4. PastDateException
5. RefundException

Además, las comprobaciones relacionadas con las reservas se concentran en componentes específicos como ReservationRulesValidator.

Esto permite controlar de forma explícita situaciones excepcionales y evita que las reglas de negocio dependan únicamente de la interfaz de usuario.

10.4. Rendimiento

Escenario:

Los usuarios consultan información de pistas, reservas o comentarios y aplican filtros y ordenaciones sobre los resultados.

Respuesta arquitectónica:

La aplicación dispone de componentes específicos para el filtrado y ordenación de consultas.

Los QueryFilters permiten encapsular los criterios utilizados para consultar diferentes tipos de información:

1. CourtFilter
2. FeedbackFilter
3. ReservationFilter
4. UserFilter

Además, los modelos utilizan mecanismos comunes mediante los concerns Filterable y Sortable.

Esta organización permite reutilizar la lógica relacionada con las consultas y evita implementar los mismos mecanismos directamente en diferentes controladores.

No se establecen en este documento objetivos cuantitativos de rendimiento, ya que no se ha realizado una prueba de carga ni se dispone de un volumen de usuarios representativo.

10.5. Usabilidad

Escenario:

Un usuario interactúa con la aplicación y necesita realizar operaciones como consultar disponibilidad, crear una reserva o gestionar sus datos sin tener que recargar constantemente la página.

Respuesta arquitectónica:

La aplicación utiliza Blade como sistema de plantillas y Livewire para proporcionar componentes interactivos.

Livewire permite implementar determinadas interacciones dinámicas manteniendo la aplicación dentro del ecosistema Laravel.

La utilización de componentes específicos para formularios y acciones permite estructurar las interacciones de la interfaz y reutilizar determinados comportamientos.

10.6. Desplegabilidad

Escenario:

Se realiza un cambio en el código fuente y es necesario desplegar una nueva versión de la aplicación.

Respuesta arquitectónica:

El código fuente se encuentra en GitHub y está conectado con Render.

El despliegue utiliza Docker y un Dockerfile específico para el entorno de Render.

Esta configuración reduce el trabajo manual necesario para desplegar nuevas versiones.

Además, la base de datos PostgreSQL se encuentra externalizada en Neon, por lo que el ciclo de vida del contenedor de la aplicación está separado de la persistencia principal de los datos.

11. Riesgos y deuda técnica

En esta sección se identifican las principales limitaciones, riesgos y posibles mejoras de la arquitectura actual.

La aplicación ha sido desarrollada como proyecto de portfolio, por lo que algunas decisiones están orientadas a mantener una infraestructura sencilla y no a cubrir las necesidades de un sistema de producción con una carga elevada o requisitos de alta disponibilidad.

11.1. Configuración del correo electrónico

El sistema de correo de Laravel está configurado actualmente mediante:

```
MAIL_MAILER=log
```

Por tanto, los correos generados por la aplicación se registran en los logs en lugar de enviarse a los usuarios mediante un proveedor de correo electrónico.

Esta configuración no sería adecuada para un entorno de producción en el que los usuarios deban recibir notificaciones por correo.

Habría que configurar un proveedor de correo electrónico real mediante SMTP o mediante un servicio especializado. La utilización de las clases Mailable de Laravel permite realizar este cambio principalmente mediante configuración, manteniendo separada la definición de los mensajes de su mecanismo de transporte.

11.2. Almacenamiento local del contenedor

La aplicación utiliza:

FILESYSTEM_DISK=local

El servicio de Render no dispone actualmente de un disco persistente configurado.

El almacenamiento local del contenedor no debe considerarse un mecanismo de persistencia permanente.

Si en el futuro la aplicación necesitara almacenar archivos que deban conservarse entre despliegues o diferentes instancias, el almacenamiento local no sería una solución adecuada.

Habría que utilizar un sistema de almacenamiento externo, como un servicio de almacenamiento de objetos, para los archivos que necesiten persistencia.

Otra posibilidad sería utilizar almacenamiento persistente proporcionado por la infraestructura de despliegue, siempre que sus características fueran adecuadas para las necesidades de la aplicación.

11.3. Almacenamiento de sesiones

Las sesiones están configuradas mediante:

SESSION_DRIVER=file

Las sesiones se almacenan, por tanto, en el sistema de archivos del entorno de ejecución.

Esta configuración resulta sencilla para una aplicación ejecutada en una única instancia, pero puede presentar limitaciones si la aplicación se escala horizontalmente y las diferentes instancias no comparten el mismo almacenamiento de sesiones.

Si la aplicación necesitara escalar horizontalmente, podría utilizarse un mecanismo de almacenamiento compartido para las sesiones, como Redis o la base de datos.

11.4. Caché y colas mediante PostgreSQL

La aplicación utiliza:

CACHE_STORE=database

QUEUE_CONNECTION=database

Tanto la caché como los trabajos pendientes de las colas utilizan PostgreSQL.

Esta configuración simplifica la infraestructura al evitar incorporar servicios adicionales, pero hace que determinadas operaciones auxiliares compartan recursos con la base de datos principal.

En una aplicación con una carga considerable, el uso intensivo de caché o colas podría aumentar la carga sobre PostgreSQL.

Si las necesidades de rendimiento aumentasen, podría introducirse un sistema especializado para estos mecanismos, como Redis.

Para el tamaño y objetivo actual de la aplicación, la configuración existente permite mantener una infraestructura más sencilla.

11.5. Dependencia de servicios externos

La aplicación depende de diferentes servicios externos:

1. Render para el despliegue.
2. Neon para PostgreSQL.
3. Stripe para los pagos.

La disponibilidad y funcionamiento de determinadas funcionalidades dependen parcialmente de estos proveedores.

Por ejemplo, una incidencia en Stripe podría impedir realizar determinadas operaciones relacionadas con los pagos, mientras que una incidencia en Neon afectaría al acceso a los datos de la aplicación.

En un entorno de producción con requisitos elevados sería necesario definir estrategias específicas de disponibilidad, monitorización, recuperación ante errores y, cuando fuera necesario, mecanismos de contingencia.

Para el proyecto actual no se considera necesario introducir infraestructura adicional para cubrir estos escenarios.

11.6. Ausencia de objetivos de disponibilidad

La aplicación está desplegada como un Web Service en Render y no se ha diseñado una infraestructura específica de alta disponibilidad.

La arquitectura actual no pretende garantizar características propias de sistemas que requieran alta disponibilidad o tolerancia a fallos avanzada.

Si la aplicación pasara a utilizarse en un entorno con requisitos de disponibilidad elevados, podrían estudiarse mecanismos como:

1. Múltiples instancias de aplicación.
2. Balanceo de carga.
3. Monitorización avanzada.
4. Alertas.
5. Estrategias de recuperación ante fallos.
6. Copias de seguridad y procedimientos de recuperación de la base de datos.

Estas medidas no se consideran necesarias para el objetivo actual del proyecto.

11.7. Ausencia de pruebas de carga

No se han definido métricas cuantitativas de rendimiento ni se ha realizado una prueba de carga representativa de un entorno de producción.

No es posible determinar mediante mediciones cuál sería el comportamiento de la aplicación ante un número elevado de usuarios concurrentes.

Si la aplicación fuera a utilizarse en producción, podrían realizarse pruebas de carga y estrés para identificar posibles cuellos de botella y establecer objetivos cuantitativos de rendimiento.

Estas pruebas permitirían evaluar especialmente:

1. Consultas a PostgreSQL.
2. Operaciones relacionadas con las reservas.
3. Procesamiento de pagos.
4. Consultas con filtros.
5. Consumo de recursos del Web Service.

11.8. Escalabilidad

La arquitectura está orientada a una aplicación web de tamaño reducido y utiliza una infraestructura sencilla.

Si el número de usuarios o el volumen de operaciones aumentase considerablemente, determinados componentes podrían convertirse en puntos de saturación.

Entre ellos se encuentran principalmente la base de datos, el almacenamiento de sesiones, las colas y los recursos disponibles para la aplicación.

Un crecimiento significativo podría requerir:

1. Escalado horizontal de la aplicación.
2. Un mecanismo compartido para las sesiones.
3. Un sistema especializado para caché y colas.
4. Optimización de consultas.
5. Revisión de índices de PostgreSQL.
6. Procesamiento asíncrono de determinadas operaciones.

No se considera necesario implementar estas medidas mientras las necesidades del sistema no lo requieran.

11.9. Monitorización

Render proporciona mecanismos de monitorización de la infraestructura y acceso a logs del servicio.

La aplicación no cuenta actualmente con una plataforma específica de observabilidad que centralice métricas, trazas y alertas de aplicación.

Ante un uso real de la aplicación, podría resultar más difícil detectar y diagnosticar determinados problemas de aplicación o de negocio.

En un entorno de producción podría incorporarse una solución de monitorización y gestión de errores que permitiera recopilar información como:

1. Errores de aplicación.
2. Excepciones.
3. Rendimiento.

4. Eventos relevantes.
5. Métricas de uso.
6. Alertas.

11.10. Deuda técnica y mejoras futuras

Las principales mejoras identificadas para una hipotética evolución del proyecto son:

Área	Situación actual	Posible evolución
Correo	MAIL_MAILER=log	Proveedor de correo real
Archivos	Almacenamiento local	Almacenamiento persistente externo
Sesiones	file	Redis o base de datos si se escala
Caché	PostgreSQL	Redis si aumenta la carga
Colas	PostgreSQL	Sistema especializado si aumenta la carga
Monitorización	Logs e infraestructura de Render	Plataforma de observabilidad
Rendimiento	Sin pruebas de carga	Pruebas de carga y optimización
Disponibilidad	Infraestructura sencilla	Alta disponibilidad si fuese necesaria
Escalabilidad	Aplicación de tamaño reducido	Escalado horizontal

Estas cuestiones no representan necesariamente defectos de la implementación actual. Son principalmente consecuencias de haber diseñado la aplicación para un contexto de tamaño reducido y para su utilización como proyecto de portfolio.

11.11. Resumen

La arquitectura actual prioriza la simplicidad, la separación de responsabilidades y el uso de servicios gestionados frente a la incorporación de infraestructura adicional.

Las principales limitaciones identificadas están relacionadas con aspectos que serían relevantes si la aplicación evolucionara hacia un sistema de producción con mayores requisitos de disponibilidad, escalabilidad, persistencia y monitorización.

La identificación de estos aspectos permite establecer una base para futuras evoluciones sin necesidad de introducir complejidad innecesaria en la versión actual del proyecto.

12. Glosario

En esta sección se definen los principales términos utilizados en la documentación y en la aplicación.

1. Cliente: usuario registrado que puede consultar pistas, realizar y gestionar reservas, enviar comentarios y consultar el historial de comentarios.
2. Gestor: usuario encargado de gestionar las pistas, reservas y comentarios de los clientes.
3. Administrador: usuario que dispone de los permisos de un gestor y, además, puede gestionar los usuarios con rol de gestor.
4. Pista: instalación deportiva que puede ser reservada mediante la aplicación. Las pistas disponibles son Campo fútbol 7, Campo fútbol 11, Pádel, Atletismo, Multiusos, Pista interna y Pista externa.
5. Reserva: registro que representa la utilización de una pista por parte de un cliente en una fecha y franja horaria determinadas.
6. Franja horaria: intervalo de tiempo durante el cual una pista puede ser reservada.
7. Disponibilidad: conjunto de fechas y franjas horarias en las que una pista puede ser reservada.
8. Comentario: comunicación enviada por un cliente a través de la aplicación.
9. Feedback: término utilizado internamente en el código para representar los comentarios enviados por los clientes.

10. Pago: operación asociada al proceso de reserva y gestionada mediante Stripe.
11. Webhook: notificación enviada por Stripe a la aplicación para comunicar un evento relacionado con un pago.
12. Rol: tipo de usuario que determina las operaciones que puede realizar dentro de la aplicación. Los roles disponibles son: cliente, gestor y administrador.
13. Service: componente encargado de encapsular operaciones de aplicación o lógica que requiere coordinación entre diferentes componentes.
14. Policy: componente utilizado para determinar si un usuario está autorizado a realizar una determinada operación sobre un recurso.
15. Form Request: clase de Laravel utilizada para encapsular la validación de una petición HTTP.
16. Query Filter: componente utilizado para encapsular criterios de filtrado aplicados a las consultas de datos.
17. Mailable: clase de Laravel utilizada para representar y configurar un mensaje de correo electrónico.
18. Eloquent: ORM de Laravel utilizado para trabajar con los datos almacenados en PostgreSQL mediante modelos.
19. Render: plataforma utilizada para desplegar la aplicación web.
20. Neon: servicio externo utilizado para proporcionar la base de datos PostgreSQL de la aplicación.
21. Stripe: servicio externo utilizado como pasarela de pago.
22. Livewire: tecnología utilizada para desarrollar componentes de interfaz interactivos dentro del ecosistema Laravel.
23. Blade: sistema de plantillas utilizado por Laravel para generar las vistas de la aplicación.
24. Vite: herramienta utilizada para gestionar y compilar los recursos frontend de la aplicación.
25. Docker: tecnología utilizada para empaquetar y ejecutar la aplicación en un entorno reproducible.

12.1. Convenciones utilizadas en el código

Algunos términos utilizados en el código pueden no coincidir exactamente con la terminología utilizada por los usuarios de la aplicación.

En particular:

1. Feedback: se utiliza en el código para representar los comentarios enviados por los clientes.
2. Court: se utiliza en el código para representar una pista deportiva.
3. Reservation: representa una reserva.
4. User: representa a cualquier usuario del sistema, independientemente de su rol.
5. Manager: representa al usuario con rol de gestor.
6. Admin: representa al usuario con rol de administrador.

Estas diferencias se deben a que la nomenclatura interna del código utiliza determinados términos en inglés, mientras que la interfaz y la documentación funcional utilizan principalmente términos en español.